

Usage Metering & Billing Engine

Project explainer — what we built, why, and the alternatives we weighed

Contents

Project Explainer	2
0. The problem in one paragraph	3
1. How the project was actually built (the process)	4
2. Technology choices	5
3. Data model design	10
4. The metering pipeline (POST /generate)	13
5. Quota enforcement	15
6. Cost calculation	18
7. Stripe integration	20
8. The background job & reconciliation	22
9. Error handling, the API contract, and security posture	24
10. Testing strategy	26
11. Deployment & developer experience	28
12. Scope — what we deliberately left out, and why	29
13. Honest retrospective — what we'd change	30
14. How to explain this in 30 seconds and in 3 minutes	31
Appendix — stack at a glance	32

Project Explainer

flyrank-capstone-metering-billing — Usage Metering & Billing Engine

This document explains the project the way you'd explain it to a new engineer joining the team, or the way you'd defend it in an interview: **what we built, why we built it that way, what the alternatives were, and what trade-offs we knowingly accepted.**

It is deliberately **not** a code audit (that's `AUDIT.pdf`). Here the unit of discussion is a **decision**, not a file.

0. The problem in one paragraph

Every SaaS product has to answer three questions about every customer: **how much have they used, what does that cost, and have they hit their limit?** Getting this wrong is expensive in a very literal way — a retried network request that double-charges, a webhook delivered twice that flips a plan back and forth, a customer sitting exactly on their quota boundary. This service is the backend that answers those three questions **correctly* under retries, failures, and concurrency*. It meters simulated LLM token usage per tenant, enforces monthly plan quotas before each billable action, prices usage in exact integer money, and keeps subscription state in sync with Stripe through signature-verified, idempotent webhooks — with a nightly reconciliation job as a safety net for anything the webhooks miss.

The constraints we were handed (and kept):

- **\$0 stack, no credit card, ever.** Everything runs on free tools: Postgres in

Docker, Stripe **test mode**, the Stripe CLI.

- **Stripe test mode only.** Real API shapes, test cards, zero real money.
- **Money is integers.** Never floats. (More on why in §6.)
- **Intentionally small scope.** 2 plans (Free/Pro), 2 usage types (API calls + AI

tokens), 1 billable endpoint. No invoicing, no proration, no overage billing in the core — those are explicitly stretch goals.

- **A stranger can run it.** One documented command on a clean machine.

Everything below follows from those five constraints plus the correctness bar.

1. How the project was actually built (the process)

What we did. The build ran in five phases, each ending in a concrete "gate":

Phase	Output	Gate
1 — Design	schema, plans/quotas, the metering API contract, idempotency strategy, one explicit non-goal	design doc signed off
2 — Core billing	idempotent POST /generate, quota enforcement with honest status codes	the double-count test passes; boundary returns 429/402
3 — Stripe	Checkout in test mode, webhook verification + dedup, plan sync	a test Checkout flips a tenant Free -> Pro via webhook
4 — Cost & hardening	CostService, pinned pricing tests, tenant-isolation tests, header/validation hardening	/usage numbers match pinned tests; all tests green
5 — Demo prep	clean-boot automation, docs, the reconciliation stretch goal	all five Layer-2 probes pass on a fresh boot

Then a sixth pass — "rubric hardening" (one commit, 76b3615) — took an external code review and closed the gaps it found: a concurrency hole in quota enforcement, the missing "real background job", missing DB indexes, Retry-After, a too-broad exception handler, a concurrent-webhook-insert 500, fabricated Stripe dates, and doc/code drift.

Why a phased build with gates. Billing bugs are silent and costly; "it looks done" is not a safe signal. Each gate is an *observable* fact — a green test, a curl transcript, a log line — pasted into EVIDENCE.md against a checklist item. "Done" is never a feeling.

Why the coordinator files (AGENTS.md, SPECS.md, rules/, knowledge/, tech-debt-tracker.md, learnings.md, BUILDLOG.md). This was built as a "harness-engineered" project: the rules that must not be broken (integer money, idempotency discipline, quota status codes, Stripe webhook contract, test coverage bar) live in rules/ as short hard constraints; business facts (pricing, plan limits) live in knowledge/; deliberately counter-intuitive choices live in tech-debt-tracker.md so nobody "fixes" them later; generalisable lessons go in learnings.md. The point is that the *reasoning* is version-controlled next to the code, not lost in chat history.

What we could have done instead. Build it in one shot against the brief. Faster to start, but with a correctness-critical system the phase gates are what stop you from discovering at the demo that the double-count test never actually ran. The overhead of the coordinator files is real (~15 small markdown files); the payoff is that six months later the answer to "why is the quota check before the write when the diagram says after?" is one grep away (tech-debt-tracker.md), not a shrug.

2. Technology choices

2.1 Language — Python 3.12

What we did. Python.

Why. The brief allows Node or Python; the rest of the internship track is Python, so the reference material (the cost-config pattern, the webhook pattern) ports directly. Python's `stripe` SDK is first-class, and `asyncio` + `asyncpg` gives real concurrency for the parts that matter (concurrent `/generate`, concurrent webhooks) without threads.

Alternative. Node + Express. Equally valid; Stripe's Node SDK is arguably the most polished. We didn't pick it only because it would mean re-deriving patterns the track already taught in Python.

2.2 Web framework — FastAPI

What we did. FastAPI (with Uvicorn as the ASGI server).

Why.

- **Pydantic request/response models for free.** Every endpoint declares its shape as a typed class; bad input becomes a clean 422 at the boundary automatically, never a 500. This is one of the eight cross-cutting requirements ("validation at the boundary") and FastAPI gives it to you by default.
- `Header(...)` **dependency injection.** `X-Tenant-ID` and `Idempotency-Key` are just typed parameters; a missing header or a non-UUID tenant id is a framework-level 422 before our code runs.
- **Native async.** The event loop lets two `/generate` requests genuinely interleave, which is exactly the condition our concurrency tests exercise.
- `lifespan` **hook.** A clean place to start/stop the background scheduler with the app.

Alternatives.

- **Flask.** Synchronous by default; async support is bolted on. We'd lose the interleaving that makes the concurrency story testable, and we'd hand-roll request validation.
- **Django + DRF.** Heavyweight for a service with 7 endpoints and no admin UI, no templates, no user model. The Django ORM is sync-first. The batteries we'd get (admin, auth) are things the brief explicitly puts out of scope.

Trade-off accepted. FastAPI's async everywhere means every DB call is `awaited` and every test is `async def`; the `pytest-asyncio` event-loop model has sharp edges (see §11). Worth it for the concurrency guarantees.

2.3 Database — PostgreSQL 16

What we did. PostgreSQL, one container, one local volume.

Why.

- `SELECT ... FOR UPDATE` **row locks** are the mechanism we use to serialise concurrent quota checks for one tenant (§5). We need a real RDBMS with real row-level locking.
- **Unique constraints as a correctness backstop.** `(tenant_id, idempotency_key)` and

the Stripe-event-id primary key are enforced by the database, not the app, so a race between two retries still lands on exactly one row.

- `JSONB` for storing the raw Stripe webhook payload — queryable, typed, indexable

if we ever need it.

- `BigInteger` for money, so a large Pro month can't overflow 32 bits.

Alternatives.

- **SQLite.** The brief allows it. But SQLite's locking is database-level (a writer

locks the whole file), so the "two concurrent `/generate` calls" test would either serialise trivially (hiding the bug we're guarding against) or throw `database is locked`. It also has no native `JSONB` and weaker `FOR UPDATE` semantics. SQLite would have made the *hard* part of this capstone un-demonstrable.

- **MySQL.** Fine, but no upside here and slightly weaker `JSON` ergonomics than

Postgres `JSONB`.

Trade-off accepted. A single Postgres container with a file volume — no replication, no failover, no read replicas. Documented as a limitation. For a metering engine demo that's the right amount of infrastructure; production would need HA.

2.4 ORM — SQLAlchemy 2.0 (async) + asyncpg

What we did. SQLAlchemy 2.0 typed ORM models (`Mapped[...]`, `mapped_column`), the async engine, `asyncpg` as the driver.

Why.

- **2.0's typed models** read like dataclasses and give the IDE/type-checker real

column types.

- **Async engine + `asyncpg`** so DB I/O doesn't block the event loop under concurrency.
- **It's the SQL toolkit, not a framework.** We can drop to Core `select(...)` with

`func.sum / func.count` for the rollup aggregates, and use `.with_for_update()` when we need the lock — without fighting an ORM that wants to hide SQL from us.

- **Pairs with Alembic** for migrations.

Alternatives.

- **Raw `asyncpg`.** Maximum control, minimum magic. But we'd hand-write every query,

every result-to-object mapping, and lose migrations. The rollup and reconciliation logic would be a lot more code.

- **Tortoise ORM / SQLAlchemy / Piccolo.** Lighter, async-native. SQLAlchemy especially is

tempting (Pydantic + SQLAlchemy in one). We stayed on plain SQLAlchemy because it's the most battle-tested for the *locking* and *constraint* semantics this project leans on, and the track already uses it.

- **Django ORM.** Sync-first; would undercut the async story.

Trade-off accepted. SQLAlchemy async has more ceremony than a lighter ORM (session management, `expire_on_commit=False`, `await db.flush()` vs `commit()`), and the event-loop-per-test issue in the suite is partly a consequence. Accepted for the locking guarantees.

2.5 Migrations — Alembic

What we did. Two hand-written Alembic migrations (`0001_initial_tables`, `0002_indexes_and_job_runs`), applied automatically on container start.

Why. "Real persistence — schema as migrations" is a hard requirement. A migration history is also *evidence of the build* — you can see the schema evolve. Applying them in the container's start command means a clean clone is at head with zero manual steps.

Why hand-written, not `--autogenerate`. The migrations are small and we wanted them readable and reviewable (explicit `ForeignKeyConstraint`, `ondelete='CASCADE'`, `UniqueConstraint` names). Autogenerate is noisy for a five-table schema.

Alternatives.

- `Base.metadata.create_all()` **at startup**. Zero migration files. But then there's no

history, no way to evolve the schema without dropping data, and it fails the "schema as migrations" requirement. (We *do* use `create_all` — but only in the test suite, for speed and isolation. §11 explains the gap this creates.)

- **Autogenerate.** Would have been faster to write `0002`, but less readable.

Trade-off accepted. Hand-written migrations can drift from the models (they're not diffed automatically), and the test suite builds its schema from the models rather than the migrations — so a drift wouldn't fail a test. Flagged in `tech-debt-tracker` and §11; the fix is a migration-vs-metadata check in CI (which we don't yet have).

2.6 Validation & config — Pydantic v2 + pydantic-settings

What we did. Pydantic models for every request/response; `pydantic-settings BaseSettings` for config, loaded from `.env`.

Why. Validation at the boundary is a requirement, and Pydantic is how FastAPI does it. `BaseSettings` with three *required* fields (`DATABASE_URL`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`) means the process **refuses to start** if a secret is missing — fail fast, at import, not on the first request that needs it.

Alternative. `os.environ` + manual asserts, or `python-dotenv` alone. More code, no type coercion (`ENABLE_SCHEDULER` as a real `bool`), easier to forget a check.

2.7 Payments — Stripe, test mode

What we did. Stripe test mode: Checkout Sessions for the upgrade flow, webhooks for the sync-back, the Stripe CLI for local webhook delivery.

Why. The brief mandates it and it's the correct choice regardless: test mode gives real API shapes and real webhook signatures with test cards and no money movement. The *correctness* lessons (signature verification, idempotent event handling, "the database mirrors Stripe") are identical to live mode.

Why hosted Checkout, not a custom payment form. Building our own card form means handling PCI scope, card tokenisation, 3-D Secure — none of which teaches anything the capstone is about, and all of which Stripe's hosted page does for free. Our backend's job is: create a session, then react to the webhook.

Alternatives. Stripe Payment Intents / Elements (more control, more surface); a different processor (Paddle, LemonSqueezy). No reason to — Stripe test mode is free, mandated, and the reference implementation the track points at.

Trade-off accepted. We never exercise a real card charge or real 3-D Secure. For a metering-and-sync engine that's fine; the money truth lives at Stripe either way.

2.8 Background scheduler — APScheduler (in-process)

What we did. APScheduler's `AsyncIOScheduler` with a `CronTrigger` for 03:00 UTC, started from the FastAPI `lifespan` when `ENABLE_SCHEDULER` is true.

Why.

- **Zero extra infrastructure.** No broker, no worker container — it fits the \$0 stack and the "one command to run" bar.
- **Async-native.** It runs coroutines on the app's event loop, so the job can `await` the same `asyncpg` sessions the rest of the app uses.
- `CronTrigger` expresses "nightly at 03:00 UTC" directly.

Alternatives.

- **Celery + Redis/RabbitMQ beat.** The "real" answer for production: durable queue, retries, monitoring, horizontal workers. But it's a broker container + a worker container + serialization concerns — heavy for one nightly job, and it breaks the single-command promise.
- **A system cron entry running** `python reconcile_stripe.py`. Simplest possible.

But it lives *outside* the app (nothing in the repo runs it), has no retry/alert wrapper, and "add a crontab line" isn't "a stranger can run it".

- **FastAPI BackgroundTasks.** Wrong tool — those are per-request, fire-and-forget, not scheduled.
- **A cloud scheduler (GitHub Actions cron, Cloud Scheduler).** Free-ish, durable, external — a good production answer, but adds a hosting dependency the capstone doesn't want.

Trade-off accepted, and it's a real one. An in-process scheduler runs **once per replica**. Scale the API to 3 pods and the reconciliation sweep fires 3× nightly. It's also not durable — a process killed mid-job doesn't resume. Both are documented in `tech-debt-tracker` and `README`. The mitigation path is: move to an external scheduler or add a leader-election lock. For a single-instance demo, in-process is the right weight, and we made the job itself idempotent so a double-run is harmless.

2.9 Testing — pytest + pytest-asyncio + httpx

What we did. `pytest` with `asyncio_mode = "auto"` (every `async def test_just runs`), `httpx.AsyncClient` + `ASGITransport` to drive the app in-process, and `unittest.mock` to stub Stripe.

Why. Standard, fast, no network. `ASGITransport` means the tests hit the real FastAPI app (routing, validation, exception handlers) without a running server or a port. Stripe is mocked so the suite is deterministic and offline.

Alternative. Spin up the app with a live server and hit it over HTTP; use Stripe's test fixtures over the network. Slower, flakier, needs credentials. We keep that style for the *Layer-2 behavioural probes* (`verify_probes.py`) which are explicitly "run against a live instance", and keep unit/integration tests hermetic.

2.10 Packaging — Docker Compose

What we did. `docker-compose.yml` with two services (`db`, `api`), a Postgres healthcheck, `depends_on: condition: service_healthy`, and an `api` start command that runs `alembic upgrade head && python seed_tenant.py && uvicorn ... --reload`.

Why. "A stranger can run it" with one command (`docker compose up --build`). The healthcheck + `depends_on` means migrations never race a not-yet-listening Postgres. The start command means the DB is at head and has demo data before the first request.

Trade-off accepted. The shipped default is a *development* configuration: `--reload` plus a bind-mount of the source, `postgres/postgres` credentials, port 8000 hard-mapped. Great for "clone and poke at it", not what you'd deploy. A `docker-compose.prod.yml` would be the next step; we didn't write one because the capstone's success criterion is "a stranger can run and demo it", not "it's production-hardened".

3. Data model design

Six tables: `tenants`, `plans`, `subscriptions`, `usage_events`, `webhook_events`, `job_runs`.

3.1 Multi-tenancy: shared tables + `tenant_id`

What we did. One set of tables; every tenant-owned row carries a `tenant_id` foreign key to `tenants.id` (with `ON DELETE CASCADE`), and **every query filters by `tenant_id` at the query layer**, not after fetching.

Why. It's the simplest model that satisfies "customer data isolated per tenant" for this scale, it keeps migrations single, and it makes cross-tenant aggregate questions (if we ever needed them) trivial.

`tests/test_tenant_isolation.py` proves tenant A's rollup never contains tenant B's rows.

Alternatives.

- **Schema-per-tenant** (a Postgres schema per customer). Stronger isolation, but

migrations now fan out across N schemas and connection routing gets complex. Overkill for 2 demo tenants.

- **Database-per-tenant.** Strongest isolation, standard for regulated enterprise SaaS.

Operationally heavy — N databases to migrate, back up, monitor. Not remotely justified here.

Trade-off accepted. Shared-table isolation is only as good as the discipline of always scoping queries by `tenant_id`. A forgotten filter is a data leak. We mitigate with the isolation test and by funnelling all usage reads through one module (`usage_query.py`) that always takes `tenant_id`.

3.2 `plans` as a table, not an enum

What we did. A `plans` table with `id` ("free"/"pro"), `max_api_calls`, `max_tokens`, seeded on startup.

Why. Quota limits are *data*, and putting them in a row means the quota check is a join, not a hardcoded `if plan == "free": 1000`. Adding a third plan later is an `INSERT`, not a code change. It also models reality — Stripe has Products/Prices; our `plans` is the local mirror of "what a tier allows".

Alternative. A Python enum / dict of limits. Fewer moving parts, one less table to seed. But then the limits live in code, the quota service has a branch per plan, and a pricing/limit change is a deploy. For a system whose whole point is "limits and money", limits-as-data is the honest model.

Trade-off. We now have to seed the `plans` table (handled, idempotently, in `seed.py` and on container start) and every environment must run the seed. Minor.

3.3 `usage_events` is an append-only event log

What we did. Every billable action writes one **immutable** `usage_events` row: `tenant_id`, type ("ai_token" or "api_call"), quantity, idempotency_key, the four per-category token counts, `cost_microcents`, `created_at`. Nothing is ever updated. "Current usage this month" is computed by aggregating rows in a time window.

Why.

- **Auditability.** You can reconstruct exactly what was charged, when, and under which

idempotency key. A running counter can't tell you *why* it's at 4,213.

- **Idempotency lives naturally on the row.** The (`tenant_id`, `idempotency_key`) unique

constraint is on the event itself, so "was this request already recorded?" is a primary-key-style lookup.

- **Correct billing-window math.** `created_at >= period_start` gives you the current

cycle's usage; last cycle's rows are still there for history.

Alternatives.

- A `usage_counters` row per (tenant, month) that you `UPDATE ... SET n = n + q.`

Reads are $O(1)$ instead of an aggregate. But: you lose the audit trail, idempotency needs a *separate* table anyway (you can't dedupe a counter increment), and the increment is a hot row that every request contends on. You'd likely end up keeping the event log *as well*, for audit — so now you maintain two sources of truth.

- **Event sourcing with a projection/materialised view.** The counter, kept fresh by a

trigger or a background projector. This is the "best of both" and a reasonable next step if read volume grew. We didn't need it: the aggregates are indexed (`ix_usage_events_tenant_type_created`) and the scope is small.

Trade-off accepted. Every quota check and every `GET /usage` runs 2–4 aggregate queries instead of one indexed row read. At capstone scale that's microseconds; at scale you'd add the projection. We chose the simple, auditable model and made the index match the query exactly — and then measured that "microseconds" claim instead of asserting it: `bench/index_benchmark.py` seeds 100,000 rows across 40 tenants and times the exact aggregate query with and without `ix_usage_events_tenant_type_created`: **6.25ms mean -> 0.45ms mean, ~13x faster.** Full transcript: `bench/RESULTS.md`.

3.4 One event per `/generate` call (not two rows)

What we did. `POST /generate` writes exactly one row, `type="ai_token"`, whose `quantity` is the sum of the four token categories. It never writes a separate `type="api_call"` row. "API calls used" is then computed as `*SUM(quantity) of api_call rows + COUNT of ai_token rows*`.

Why. The brief says the billable endpoint "creates *a* usage event" (singular). The two "usage types" are two *quota dimensions*, not a mandate for two rows. One row that counts as "1 API call and N tokens" is the minimal honest model, and it keeps the (`tenant_id`, `idempotency_key`) unique constraint simple (two rows with the same key would force `type` into the constraint).

Alternative. Emit both an `api_call` row and an `ai_token` row per call. Cleaner in the sense that "API calls used" becomes a plain `COUNT(*) WHERE type='api_call'`. Costs: the unique constraint becomes (`tenant_id`, `idempotency_key`, `type`), every write is two INSERTs, and the replay/idempotency logic has to handle "one of the two rows exists".

Trade-off accepted / known weak point. Because the endpoint never emits an `api_call` row, that code path (`type="api_call"` in `CostService`, `UsageQuery`) is exercised only by tests, not by the running app. A reviewer will read it as unused. It's defensible as "a bulk-metering hook, tested and ready", but honestly it should either be wired to a bulk endpoint or removed and `api_calls_used` simplified to `COUNT(*)`.

3.5 subscriptions — one per tenant, mirrors Stripe

What we did. `subscriptions.tenant_id` is `UNIQUE` (one subscription per tenant). It stores `stripe_subscription_id` (unique, nullable), `stripe_customer_id`, `plan_id`, `status`, and `current_period_start/end` (both nullable).

Why one per tenant. The scope is 2 plans, one active tier at a time. A tenant is either Free or Pro; there's no concept of stacked subscriptions or add-ons.

Why `current_period_*` nullable. Stripe doesn't always hand us period boundaries (they moved location across API versions, and some event shapes omit them). Rather than *fabricate* a window (`now .. now + 30d`), which would silently corrupt the quota period and the `Retry-After` math, we store `NULL` and fall back to "the current calendar month" in the quota/rollup layer. `NULL` is honest; a fabricated date is a latent bug.

Why keep stripe_customer_id on downgrade. When a subscription is cancelled we set `plan_id="free"`, `status="active"`, `stripe_subscription_id=NULL` — but we **keep** the customer id, so a future re-upgrade reuses the same Stripe customer instead of creating a duplicate.

Alternative. A `subscriptions` table that allows history (multiple rows per tenant, one "current"). More faithful to Stripe, needed if you ever support plan history / churn analytics. Out of scope here; one mutable row is enough.

3.6 `webhook_events` — the Stripe event id **is* the primary key*

What we did. `webhook_events.id` is a `VARCHAR(255)` primary key holding Stripe's `evt_...` id. `type` and the raw payload (JSONB) are stored alongside.

Why. Webhook idempotency and the "have we seen this event?" ledger are the **same object**. Dedup is a primary-key lookup (`db.get(WebhookEvent, event.id)`), and a concurrent second delivery can't insert because the PK collides — the same correctness-by-constraint pattern as `usage_events`.

Alternative. An auto-increment PK plus a separate unique index on the event id. Functionally identical; using the event id **as** the PK is just more direct and makes the intent obvious.

Why store the whole payload. Debuggability and replay. If a sync goes wrong you have the exact bytes Stripe sent. JSONB so it's queryable later if needed.

3.7 `job_runs` — a durable execution ledger

What we did. Every background-job run writes one `job_runs` row: `job_name`, `status` (`running` -> `success` | `failed`), `attempts`, `started_at`, `finished_at`, and `error` (the traceback string on failure).

Why. The "background job" requirement includes "failure alert". A CRITICAL log line is one half of that; a **queryable** record is the other. `GET /admin/jobs` reads this table so a human (or a monitor) can see "did last night's reconciliation run, did it retry, did it fail, and why" without grepping logs. It also makes the job demoable.

Alternative. Logs only. Cheaper (no table, no migration). But logs are ephemeral and un-queryable; "show me the last 5 reconciliation runs" becomes a log-aggregation problem. For a system that's **about** correctness and reliability, a small durable ledger is worth one table.

Known gap. A process killed mid-job leaves a row stuck at `status="running"` forever. There's no startup reaper. Detectable (`finished_at IS NULL` and old), not yet handled.

3.8 Money as `BigInteger` *micro-cents*

Covered in depth in §6. Short version: `cost_microcents` is a `BigInteger` because a big Pro month can exceed 2^{31} micro-cents, and it's an integer because floats lose money.

4. The metering pipeline (`POST /generate`)

4.1 Simulated tokens, not a real model call

What we did. The request body carries an optional `mock_usage` object with the four token counts. No LLM is called. If `mock_usage` is omitted, the call still counts as 1 API call and 1 token.

Why. The brief is explicit: "You're metering numbers, not calling a model — no AI key needed at all." Wiring a real provider would add cost, a secret, latency, and non-determinism to tests — for zero credit, because the capstone is about the **metering and billing correctness**, not the generation.

Alternative. Call a real model and count its returned usage. More "realistic", but it makes every test non-deterministic and needs a paid key. Explicitly out of scope, and logged in `tech-debt-tracker` as "do not wire in a live provider".

4.2 Idempotency: three layers, on purpose

What we did. A billable request carries a client-supplied `Idempotency-Key` header. `MeterService.record` guarantees exactly-once with **three** mechanisms:

1. **An in-request pre-check** `SELECT` for `(tenant_id, idempotency_key)`. Hit -> return

the existing row, no write.

2. **A database** `UNIQUE (tenant_id, idempotency_key)` **constraint**. The `INSERT` is

attempted and the DB is the arbiter.

3. **An** `IntegrityError` -> **rollback** -> **re-query** fallback. If the `INSERT` lost a race, we

read the winner's committed row and return **that**, so both callers get identical results and there's still one row.

The `/generate` handler **also** does a pre-check of its own, so a simple retry (client re-sends after a timeout) returns the original response body without even entering the service.

Why all three, not one:

- **The DB constraint alone** would work for correctness but every retry would be a

failed `INSERT` + `rollback` + `re-query` — wasteful for the overwhelmingly common "retry seconds later" case, and it can't cheaply return the **original response**.

- **The app pre-check alone** has a time-of-check/time-of-use gap: two truly concurrent

requests both `SELECT` (miss), both `INSERT`. Only the DB can break that tie.

- **Layer 3** is what turns the race **loser** from a 500 into a success.

`rules/idempotency.md` forbids "simplifying" this: "the pre-check is a fast path; the constraint is the correctness guarantee. Both stay." Proven by `test_meter_service_deduplication_sequential` and `test_meter_service_concurrent_race_condition`.

Alternatives.

- **A distributed lock (Redis** `SETNX` **on the key)**. Works, but adds Redis to the stack

(against \$0-simplicity), and you still want the DB constraint as a backstop if Redis is down — so you'd have **four** layers.

- **An** `idempotency_keys` **table separate from** `usage_events` (store key -> response

blob). This is what Stripe itself does. It's more general (works for endpoints that don't create a single row). We didn't need the generality: every billable action here *is* exactly one `usage_events` row, so the key can live on that row.

Trade-off accepted. Three layers is more code than one. It's the right amount for a system where a duplicate is a double charge.

4.3 Quota check **before* the write (not the brief's order)*

What we did. `/generate` does: idempotency pre-check -> parse tokens -> **quota check** -> **record the event** -> commit.

Why. The brief's architecture sketch shows "store `usage_event` -> then quota check". We deliberately inverted it: a request that's going to be rejected should **not** leave a row behind. Storing-then-rejecting means either you write a row and then delete it (two writes, a window where usage looks inflated) or you write a row that "doesn't count" (now every aggregate needs a `WHERE rejected = false`).

`MeterService.record` is *still* independently idempotent, so it's safe to call from any order; we just don't call it for a doomed request.

Trade-off. We diverge from the reference diagram, which a literal-minded reviewer might flag. Mitigated by documenting the divergence and its reason in `rules/idempotency.md` and `tech-debt-tracker.md`.

5. Quota enforcement

5.1 The boundary rule: exactly-at-limit is allowed

What we did. A request is allowed iff `current_usage + requested_usage <= limit`. So the request that brings a Free tenant to **exactly** 100,000 tokens succeeds; the next one fails.

Why. "Boundary honesty" is one of the three genuinely hard parts of this capstone. We picked the interpretation that a customer gets to **use every unit they paid for** — 1,000 of 1,000 calls is allowed; call 1,001 is not. It's the least surprising reading of "you have 1,000 calls".

Alternatives.

- **Reject at the limit** (`current + requested < limit`). "You have 1,000" would then

mean you can make 999. Surprising and slightly stingy.

- **Allow one over / soft limit.** Useful for UX (don't fail the request that crosses

the line, just flag it), and it's basically "overage billing" — which the brief lists as a stretch goal, deliberately not in core.

The rule is stated once in `rules/quota-and-status-codes.md` and applied identically to both the API-call and the token dimension. Tests pin all three cases: just under, exactly at, one over.

5.2 402 vs 429 — two different failures

What we did.

- **402 Payment Required** — the **plan itself** doesn't permit the action: there's no subscription, or its `status` isn't `active` (canceled, past_due, ...), or it points at a missing plan.
- **429 Too Many Requests** — the plan permits it in general, but the tenant has used

their monthly allowance.

Every error body is the same shape: `{"error", "message", "code"}`, with the message stating the limit, the current usage, and the amount requested — so a machine caller can tell **which** failure happened and **why** without querying the database.

Why split them. They demand different customer actions. 402 -> "upgrade / fix your card". 429 -> "wait for the reset, or move to a bigger plan". Collapsing both into one code (or into a generic 400) throws away that signal. The brief calls this out specifically: "Status codes are how machines read your answers."

What we could have added. A 402 specifically for a **Free** tenant who hits the Free quota, nudging them to upgrade (right now that's a 429). We kept it as 429 because they **have** used their allowance — the honest code — and 429 is an acceptable answer per the probe spec. Adding an "upgrade to continue" variant would be a nice UX touch.

5.3 The concurrency problem and `SELECT ... FOR UPDATE`

The problem. Two `/generate` requests for the same tenant, both arriving at 999 of 1,000 calls. Without coordination: A reads `current = 999`, B reads `current = 999` (A hasn't committed), both compute `999 + 1 = 1000 <= 1000`, both pass, both `INSERT` -> the tenant lands at **1001**. The idempotency constraint doesn't help — the two requests have **different** keys; they're two legitimate distinct actions.

What we did. `QuotaService.check_quota` starts with `SELECT ... FROM subscriptions WHERE tenant_id = $1 FOR UPDATE`. That takes a row-level write lock on the tenant's single subscription row, held until the

request transaction commits (i.e. through the `MeterService` insert and the route's `commit()`). Request B's identical locking `SELECT blocks in Postgres` until A commits; when B proceeds it re-reads under `READ COMMITTED`, sees A's row, computes $1000 + 1 = 1001 > 1000$, and is rejected with 429 — B never inserts.

Proven by `test_quota_boundary_is_race_safe`: two concurrent attempts against a tenant pre-filled to 999 -> exactly one 200, exactly one 429, final count exactly 1000.

Measured, not just proven once. `bench/race_condition_benchmark.py` runs that same scenario 40 times against a copy of the check with `.with_for_update()` removed (the exact pre-hardening code path), then 40 times against the real, locked `QuotaService.check_quota`, and counts how often the tenant ends up over its limit:

```
BEFORE (no FOR UPDATE lock): 39/40 trials overcounted the 1000-call boundary (97.5%)
AFTER  (FOR UPDATE lock): 0/40 trials overcounted the 1000-call boundary (0.0%)
```

97.5% -> 0%. That number — not "we added a lock, trust us" — is what we'd put in front of an evaluator or a resume line, and it's what we'd point to if someone asked "how do you know the fix actually fixed it." Full transcript: `bench/RESULTS.md`.

Alternatives we considered:

- **A Postgres advisory lock** (`pg_advisory_xact_lock(hashtext(tenant_id))`). Same

effect, doesn't require a real row to lock, releases on transaction end. Slightly less obvious in the code; we preferred locking the actual subscription row because it's self-documenting ("I'm serialising on this tenant's plan state").

- **An atomic conditional insert** —

`INSERT ... SELECT ... WHERE (SELECT count(*) ...) < limit`. No lock held across the request, the DB does the check-and-insert in one statement. More elegant, but the boundary logic (two dimensions, plan join, the exact message with current/limit numbers) gets awkward to express in one SQL statement, and you lose the clean "raise a typed exception with a helpful message" path.

- **A per-tenant counter row with** `UPDATE ... SET n = n + q WHERE n + q <= limit`

`RETURNING n`. Fast, atomic, no long lock. But it reintroduces the counter (§3.3) and its downsides — no audit trail, monthly reset logic, a hot contended row.`

- **SERIALIZABLE isolation + retry on serialization failure.** Correct, database-level,

no explicit locks. But it turns some concurrent requests into 40001 errors the app must catch and retry, which is its own complexity, and it can hurt throughput more broadly than a targeted row lock.

- **Just accept the race.** A tenant occasionally goes a few units over quota. For some

products that's fine (you reconcile at invoice time). For a capstone whose headline is "proven no-double-count / correct under concurrency", accepting the race would be conceding the hardest point.

Trade-off accepted — and this is the thing to be ready to defend. The lock **serialises every /generate for a given tenant** for the full duration of the request (including the two usage aggregates and the insert). One very active tenant now pushes all its generate traffic single-file through one row lock. That's a deliberate trade: **per-tenant write throughput for correctness** — the right call for billing, but you should be able to say (a) it only serialises *within* one tenant, not across tenants, (b) the aggregates are indexed so the critical section is short, and (c) if a tenant's QPS ever made this a bottleneck, the fix is the atomic-counter approach above.

5.4 Retry-After on 429

What we did. Every 429 carries a `Retry-After` header — integer seconds until the quota window resets (the subscription's `current_period_end` if it's in the future, otherwise the first instant of next calendar month).

Why. MDN's guidance for 429 (and the brief's Phase-2 reading) is to include it. It lets a well-behaved client back off correctly instead of hammering.

What we could improve. The calendar-month fallback is coarse; a real system would track the tenant's actual billing anchor day.

5.5 Computing usage from events every time

What we did. Both the quota check and `GET /usage` compute "used so far this period" by aggregating `usage_events` in the window, through one shared module (`usage_query.py`) so the two can never disagree.

Why one shared module. Before the hardening pass, `QuotaService` and `RollupService` each had their own hand-rolled aggregates — a latent bug where enforcement and reporting could drift. Consolidating them means "what the quota check counts" is **by construction** what `/usage` shows.

Why recompute, not cache. Covered in §3.3 — auditability and correctness over a micro-optimisation the scope doesn't need. The index makes it cheap.

6. Cost calculation

6.1 Integer money — micro-cents, never floats

What we did. All money is an integer count of "micro-cents": $1 \text{ micro-cent} = 1e-4 \text{ cents} = 1e-6 \text{ USD}$; $10,000 = 1 \text{ cent}$; $1,000,000 = 1 \text{ USD}$. Stored in a `BigInteger` column. Token rates are small integers (10, 2, 30). Conversion to a human dollar string, if it ever happens, is at the presentation edge only — and in practice `GET /usage` just returns the integer.

Why.

- **Floats lose money.** $0.1 + 0.2 \neq 0.3$ in IEEE-754. Sum millions of per-token

charges and the drift is real dollars, and it's non-deterministic across platforms. The brief makes this a hard constraint; `rules/money-math.md` says any float in a money value is "a bug, full stop".

- **Sub-cent precision.** A single token can cost a tiny fraction of a cent. Cents as

the unit would force rounding on every token; micro-cents give headroom so rounding only happens (if ever) at the very end.

- `BigInteger` because Pro is 10,000,000 tokens/month; at 30 micro-cents that's

300,000,000 per category per row, aggregated over a month -> past 2^{31} .

Alternatives.

- `Decimal`. Also exact. But it's slower, it's a non-primitive that leaks into every

signature, JSON-serialising it needs care, and you *still* have to decide a scale. Integer minor units is the standard fintech answer (Stripe, ledgers) precisely because it's boring and total.

- **Store cents (integer), round per token.** Simpler unit, but you round 4× per request

and the rounding error compounds. Micro-cents pushes rounding to the edge.

Naming caveat (honest). "micro-cents" is a slight misnomer — 10,000 per cent makes the unit micro-*dollars**, not the SI 10⁶ "micro". We kept the name for API-contract and evidence-file stability and documented the exact definition in three places. If starting over, `cost_micros` (micro-USD) would be the cleaner name.

6.2 Four categories, priced separately, summed *after**

What we did. `CostService.price` computes `input·10 + cached·2 + output·30 + reasoning·30` — each category multiplied by its own rate, then added. Never "add the token counts, multiply by one rate".

Why. The categories genuinely price differently:

- **Cached input is 5× cheaper than fresh input** (2 vs 10) — the provider already

had those tokens, so it discounts them.

- **Reasoning ("thinking") tokens bill at the *output* rate** (30) — they're not free,

not a separate discounted tier; they're work the model did, billed like output.

- Because the rates differ, `(input + cached + output + reasoning) · rate` is wrong for

any single rate on a mixed request.

This mirrors how real providers (the brief points at Gemini's pricing page) structure token billing.

`rules/money-math.md` and `knowledge/pricing-plan.md` state the rule; `test_cost.py` pins each category in

isolation *and* the combined worked example (209,000 micro-cents for a specific mix), and `test_metering.py` asserts the same number end-to-end.

6.3 Constants pinned in config + covered by tests

What we did. The five rates live in one module, `app/config/pricing.py`, with a comment explaining the unit. `test_cost.py` asserts exact expected totals (not ranges).

Why. "Pricing constants pinned in config and covered by tests" is a hard requirement, and it's good practice: a pricing change is a one-file diff plus a test update, and the test *forces* you to hand-compute the new expected number — if you can't, you don't understand the change yet.

6.4 `API_CALL_RATE = 0` and the plan fee

What we did. API calls have a flat, count-based quota but a **\$0 per-call metered rate**. The recurring plan fee (\$49/mo for Pro) lives at Stripe and is **not** folded into the `cost_microcents` that `GET /usage` returns — that figure is *metered usage* cost only.

Why. The capstone's model is "2 usage types, 1 billable endpoint". API calls are metered for *quota*, not for a per-call charge; the money for "being on Pro" is the subscription fee, which is Stripe's job to collect. Mixing the subscription fee into the usage rollup would conflate two different things (what you owe for *usage* vs what you owe for *access*).

What we could have done. Add a `plans.monthly_fee_microcents` column and have `GET /usage` return `{usage_cost, plan_fee, total}`. It'd be a more complete "what will my invoice look like" answer. We left it out because invoicing is explicitly a stretch goal, and documented the omission in `README` limitations so it doesn't read as a bug.

6.5 Price at write time, not read time

What we did. `cost_microcents` is computed by `CostService` when the `usage_events` row is inserted and stored on the row. `GET /usage` and the reconciliation summary *sum the stored values*; they don't re-run pricing.

Why. A price change should not retroactively re-cost last month's usage. Pinning the cost at the moment of use is how ledgers work — the number is a historical fact.

Trade-off. If pricing had a bug, fixing the constant doesn't fix already-written rows; you'd need a backfill. Acceptable — that's true of any ledger, and it's the safe direction (you never silently change a past charge).

7. Stripe integration

7.1 The Checkout flow

What we did. `POST /checkout` creates a Stripe Checkout Session (`mode="subscription"`) for the Pro price and returns its URL. The customer pays on Stripe's hosted page. Stripe then delivers webhooks; our backend reacts to those.

Why hosted Checkout. It's the least code for the most coverage — Stripe handles the card form, SCA/3-D Secure, receipts, tax if configured. Our backend's only jobs are "start a session" and "react to the result". Building a custom flow with Payment Intents would mean owning the card UI and its edge cases for no capstone credit.

7.2 `tenant_id` in the metadata — twice

What we did. When creating the session we put `metadata: {tenant_id, plan_id}` and `subscription_data.metadata: {tenant_id, plan_id}`.

Why. `metadata` lands on the `*Checkout Session*` object (readable from the `checkout.session.completed` event). `subscription_data.metadata` is copied by Stripe onto the `*Subscription*` object (readable from `customer.subscription.updated / .deleted` and from a `Subscription.retrieve`). Putting it in both means **every downstream event can identify the tenant from the payload alone**, with a DB lookup only as a fallback.

Alternative. Store a `checkout_session_id -> tenant_id` mapping in our DB at session creation, and look it up when the webhook arrives. Also works; it's a round-trip and a row we don't need if the metadata rides along.

7.3 Verify the signature first, against the raw body

What we did. The webhook handler reads the raw request bytes, calls `stripe.Webhook.construct_event(raw_bytes, signature_header, webhook_secret)` **before touching the payload**. A bad or tampered signature -> 400, and **no code past that line runs** — the payload is never parsed, never stored, nothing changes. A malformed-JSON body is also caught -> 400.

Why raw bytes. The signature is an HMAC over the exact bytes Stripe sent. If you parse to JSON and re-serialise, whitespace/key-order differences break the HMAC. FastAPI lets us get `await request.body()` before any parsing.

Why first. A forged payload should cost us nothing — no DB write, no processing. Verification is the gate; everything else is behind it.

7.4 Idempotent event handling

What we did. After verification: `db.get(WebhookEvent, event.id)` — if we've seen this event id, return `{"status": "success", "message": "duplicate"}` with **200** (so Stripe stops retrying). Otherwise insert the `webhook_events` row, apply the change, and `commit()` — and if that commit hits an `IntegrityError` (a **concurrent** second delivery racing past the `db.get`), catch it, roll back, and return "duplicate" too.

Why 200 on a duplicate. Stripe retries any non-2xx. A replay is not an error — it's expected — so we ack it.

Why both a `db.get` check and an `IntegrityError` catch. Same reasoning as `/generate`: the pre-check is the fast path, the primary-key collision is the correctness guarantee under true concurrency. Before the hardening pass, two concurrent duplicate deliveries could both pass `db.get` and one would 500 on commit; now it's a clean "duplicate".

Why commit the `webhook_events` row and the subscription change together. One transaction -> you never get "recorded as processed but not actually applied", or vice versa.

7.5 Re-fetch the subscription in `checkout.session.completed`

What we did. The `checkout.session.completed` event carries a thin session object. We take the subscription id from it and call `stripe.Subscription.retrieve(...)` to get the authoritative status and period dates, then sync *that*.

Why. The event payload is a snapshot and can be partial; the canonical object is what `retrieve` returns. One extra API call buys certainty about what state we're mirroring.

7.6 The database mirrors Stripe — Stripe is the source of truth

What we did. Every billing-driven write to subscriptions goes through one service (`SubscriptionSync`), which is only ever called from (a) the verified webhook handler and (b) the reconciliation job — both of which take their inputs *from Stripe*. There is no endpoint that lets a client set their own plan.

Why. Payment truth lives at Stripe. If our DB and Stripe ever disagree, Stripe wins. Modelling the local row as a *cache of Stripe*, updated only through verified events, means our state can always be rebuilt from Stripe — which is exactly what the reconciliation job does.

7.7 Pro price discovery vs. a configured price id

What we did. `get_or_create_pro_price()` prefers `settings.STRIPE_PRO_PRICE_ID` if set (zero API calls); else a process-level cache; else it discovers-or-creates the "Pro Plan" product and a \$49/mo price in Stripe and caches the id.

Why the fallback exists. So a fresh clone with empty Stripe config can still run the Checkout flow end-to-end in a demo without the operator first hand-creating a product in the dashboard.

Why the config path is preferred. In any real setup you define the price *once* in the Stripe dashboard and pass its id as an env var. Listing products on every checkout (the discovery path) is wasteful and fragile (it matches on the product *name*).

Trade-off. The \$49 and `usd` are hardcoded in the discovery path. Fine for a demo default; a real deployment sets `STRIPE_PRO_PRICE_ID` and never hits that code.

7.8 What we deliberately did *not* build in Stripe

Proration on mid-cycle changes, invoice generation/PDF, emailing invoices, usage-based (metered) Stripe prices, multiple plans/add-ons. All are either explicit stretch goals or out of the "2 plans, 1 endpoint" scope. Listed as non-goals in `capstone.yaml` and `README` so they read as *choices*, not oversights.

8. The background job & reconciliation

8.1 Why a reconciliation job exists at all

The problem. Webhooks get missed — a deploy during delivery, a transient 500, Stripe giving up after its retry window, a bug that 200s an event it didn't actually apply. When that happens, our subscriptions mirror silently diverges from Stripe: a tenant who upgraded is still Free locally, or a tenant who cancelled is still Pro locally (revenue leak / access leak).

What we did. A nightly job lists *every* subscription on the Stripe account and repairs any drift: if Stripe says a tenant is Pro/active and we say Free, we fix it; if we hold a `stripe_subscription_id` that Stripe no longer lists as active (and a retrieve confirms it's canceled / a 404), we downgrade that tenant to Free. It returns a summary (fetched, synced, already_in_sync, downgraded, skipped).

Why a full compare, not incremental. The whole point is to catch what the incremental path (webhooks) missed. Comparing everything is the only way to find "an event we never received".

8.2 The generic `run_job` wrapper

What we did. `run_job(job_name, body, attempts=3, base_delay=2.0):`

- Inserts a `job_runs` row `status="running"`.
- Runs body up to `attempts` times, with linear back-off (2s, 4s) between failures.
- On success: flips the row to `success`, records which attempt won, returns the result.
- On final failure: flips the row to `failed`, stores the full traceback, and logs

CRITICAL "JOB FAILURE ALERT: ...".

- **Never raises.**

Why "never raises". It runs inside the scheduler's event loop. An exception that escaped could kill the scheduler thread silently — the worst failure mode for a reliability feature. So it catches everything, records it, alerts, and returns `None`.

Why retries + back-off. Reconciliation talks to the Stripe API; a transient network blip shouldn't mean "no reconciliation until tomorrow night". Three tries over ~6s covers the common transient case.

Why a `job_runs` row *and* a log line. The log line is the alert (a log-based monitor can page on `JOB FAILURE ALERT`); the row is the queryable history (`GET /admin/jobs`). Two audiences, two surfaces.

Why the `session_factory` parameter. The default is the app-wide session maker; tests inject a factory bound to the per-test engine so the job's sessions and the test's assertions share one database and one event loop (dodging the `asynpg` loop-mismatch class of bug — see §11).

What we could have done. Reach for an existing job framework (Celery, RQ, Dramatiq, `arg`). They give retries, dead-letter queues, and monitoring out of the box — but also a broker and a worker process. For one nightly job on a \$0 stack, a ~90-line wrapper that does retries + a durable record + an alert is the proportionate answer. If a second and third job appeared, that calculus flips.

8.3 Three ways to run it

Path	Retries?	Records <code>job_runs</code> ?	Purpose
Nightly <code>CronTrigger(03:00 UTC) -> run_job</code>	yes (≤ 3 + back-off)	yes	the real safety net

Path	Retries?	Records <code>job_runs</code> ?	Purpose
POST <code>/admin/jobs/reconcile</code> -> <code>reconcile_from_stripe</code> directly	no — single-shot	yes (hand-rolled)	demo / ops, "run it now"
python <code>reconcile_stripe.py</code> (CLI)	no	no	manual / scripting

Why the admin endpoint. So a demo or an evaluator can trigger the sweep and inspect the result without waiting until 03:00 UTC, and so you can **show** the reliability story rather than describe it.

The caveat. `/admin/jobs*` has **no authentication** — same posture as the rest of the service (§9). Anyone who can reach the service can list job history and trigger a Stripe-API-hitting sweep. Documented; in production these sit behind an admin role.

8.4 Keeping the scheduler out of tests

Three stacked defenses: `conftest.py` sets `ENABLE_SCHEDULER = False` before the app is imported; the `lifespan` hook only imports `APScheduler` **inside** the `if ENABLE_SCHEDULER` branch; and `httpx.ASGITransport` doesn't run lifespan events anyway. The reconciliation job's **name constant** lives in `reconciliation_service.py` (not `scheduler.py`) so the admin endpoint can import it without dragging `APScheduler` into the request path. This is deliberate: the test suite should never start a real scheduler.

9. Error handling, the API contract, and security posture

9.1 One error envelope, typed domain exceptions

What we did. Every error response is `{"error", "message", "code"}`. Business failures are raised as **typed domain exceptions** deep in the services (`QuotaExceededException`, `PaymentRequiredException`, `InvalidUsageError`) and translated to HTTP by handlers registered once in `app/api/errors.py`. FastAPI's validation errors (422) are wrapped into the same envelope.

Why not just raise `HTTPException(...)` everywhere. The services shouldn't know about HTTP — that's the layering rule. A service raises "quota exceeded" as a *domain* fact; the HTTP layer decides that's a 429 with a `Retry-After`. It also keeps the response shape uniform without repeating the dict at every raise site.

9.2 Why we removed the blanket `ValueError` handler

What we did (in the hardening pass). There used to be a catch-all `app.add_exception_handler(ValueError, -> 400)`. We replaced it with a narrow `InvalidUsageError(ValueError)` and a handler only for *that* type.

Why. A catch-all `ValueError -> 400` means **any** stray `ValueError` anywhere in the stack — a genuine bug, a bad `int()` cast, a library internal — gets laundered into a client-facing "400 Bad Request". The bug becomes invisible; the client gets blamed. Now only the explicit, intentional "this input is invalid" subclass maps to 400; a real bug that raises `ValueError` correctly surfaces as a 500 (which is what you want to see and fix). `InvalidUsageError` subclasses `ValueError` so existing `pytest.raises(ValueError)` call sites still pass.

The general lesson. Broad `except`/handlers that turn errors into "success-ish" responses are how bugs hide. Catch narrowly; let the unexpected be loud.

9.3 The status-code catalogue (and why each)

- **400** — blank/whitespace `Idempotency-Key`; an *explicit* all-zero `mock_usage`

(a client mistake, vs an omitted field which defaults to 1); webhook bad signature/JSON.

- **402** — no / non-active subscription / unknown plan.
- **404** — a well-formed tenant UUID that doesn't exist.
- **422** — missing required header, non-UUID tenant id, negative token field. Framework

validation, wrapped in our envelope.

- **429** — quota exceeded, with `Retry-After`.
- **502** — Stripe checkout-session creation failed (we log the real error, return a

generic message — don't leak Stripe internals to the caller).

- **500** — anything genuinely unexpected. **By design** — not laundered into a 4xx.

9.4 Security: header-only identity — a documented, deliberate limit

What we did. A caller's tenant identity is the `X-Tenant-ID` header value, full stop. No bearer token, no API key, no session. `/admin/*` has no auth at all.

Why. "Implementing real authentication/authorization" is an **explicit non-goal** in `capstone.yaml` and `README`. The capstone's difficulty is concentrated in metering correctness, money math, and Stripe — not in auth, which the intern built in a separate assignment. Spending the budget on JWT/API-keys would be gold-plating a corner the brief told us to leave.

What it actually means (stated plainly, not hidden). `docs/architecture.md` has a "Security scope (documented decision, not an oversight)" section: anyone who knows a tenant's UUID can read that tenant's `/usage` and act as them against `/generate` and `/checkout`. `tests/test_tenant_isolation.py` proves data doesn't leak *between* correctly-addressed tenants — it does **not** prove a caller is who they claim to be.

Mitigations that are present. Tenant UUIDs are 122 bits of entropy (not guessable); the realistic risk is a UUID *leaking* (a log, a URL) and then being replayable forever with no revocation. Quota caps bound the blast radius of griefing.

What we'd add first if this went past the capstone (and `docs/architecture.md` says so): per-tenant API keys (hashed, stored, revocable) or JWT verification, as a `Depends( . . . )` guard on the four core routes and everything under `/admin`.

9.5 Secrets

`STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` are required env vars, read from `.env`. `.env` is the first line of `.gitignore`. `.env.example` ships only placeholders. No handler logs a secret — error logs carry ids and exception strings, never the key values. A committed key (even a test one) is treated as an instant fail; the history is clean.

10. Testing strategy

10.1 "Done is observable"

What we did. Every checklist item is closed only with pasted evidence — a named test + its output, a curl transcript, or a log line — in `EVIDENCE.md`. There are **31 tests across 7 modules**, plus **5 Layer-2 behavioural probes** run against a live instance.

Why. In billing, "it should work" is worthless. The double-count test either exists and passes or it doesn't. The evidence file makes "done" auditable in minutes.

10.2 We test the scary cases, on purpose

The suite deliberately concentrates on the failure modes that cost money:

- **Duplicate prevention** — same key twice (sequential) and same key concurrently (the race).
- **Quota boundaries** — just under, exactly at, one over; both the API-call and token dimension; and the `*concurrent*` boundary (`FOR UPDATE`).
- **Pricing** — every category in isolation and the combined worked example, pinned to exact integers.
- **Webhooks** — forged signature -> 400 + no state change; a real event replayed -> processed once; each of the three event types -> the right subscription mutation.
- **Tenant isolation** — A can't read/affect B.
- **The job** — success records a `job_runs` row; a permanently-failing body retries N times, records `failed` with a traceback, and emits the `CRITICAL` alert.

Happy-path coverage is thin **on purpose** — a demo that shows one failure handled gracefully beats ten green happy paths.

10.3 The two concurrency tests are the crown jewels

`test_meter_service_concurrent_race_condition` (idempotency: two sessions, same key, one wins the unique index, the other falls into the `IntegrityError` branch and returns the winner's row — final count 1) and `test_quota_boundary_is_race_safe` (quota: two concurrent /generate at 999/1000, the `FOR UPDATE` lock serialises them, exactly one 200 + one 429, final count exactly 1000). These are the tests that prove the sentence "correct under retries, failures, and real-world conditions".

One of them, `test_quota_boundary_is_race_safe`, pins its subscription's `current_period_start` to `now() - 1 day` on purpose: an earlier version used `now()` and failed **intermittently in Docker** because the app container's clock and the Postgres container's clock differ by enough that DB-generated `created_at` timestamps fell just before the Python-computed window start, so the filler rows were excluded and both callers saw 0 usage. The lesson (in `learnings.md`): **never gate test data on `datetime.now()` when the row timestamps are database-generated.**

10.4 The confest design

Function-scoped async engine (fresh `drop_all` + `create_all` + re-seed per test — total isolation); `import app.models` so `create_all` sees all 6 tables regardless of collection order; and a `client` fixture that overrides FastAPI's `get_db` so route handlers use the **same** `AsyncSession` the test holds.

Why the `get_db` override. The app's global engine is created at import on one event loop; `pytest-asyncio` runs each test on a fresh function-scoped loop; a handler resolving the real `get_db` would touch a connection bound to the wrong loop -> `InterfaceError: another operation is in progress`. Three separate `learnings.md` entries document hitting this from three angles before the override settled it. This is the sharpest edge of "async everything".

10.5 The gap we accepted

The test suite builds its schema with `Base.metadata.create_all`, **not** by running the Alembic migrations. So the migrations are validated separately (`alembic upgrade head` in the container start command and by hand), but a migration that drifted from the models would not fail a test. The fix — a fixture that runs migrations against a scratch DB and diffs against `Base.metadata` — needs a CI pipeline we don't have yet.

10.6 Layer-2 probes: behavioural, against a live system

`verify_probes.py` provisions a fresh tenant and drives real HTTP against a running instance to check the five promises the evaluator will check: idempotency, quota boundary + 429, checkout -> webhook upgrade, forged/replayed webhook, pricing rollup. It hand-builds a Stripe webhook signature (HMAC-SHA256 over `"{timestamp}.{raw_body}"`) so it works even with a dummy `whsec_` secret.

Why separate from the unit tests. Unit tests prove the pieces; the probes prove the *assembled, running system* does the right thing end to end — routing, exception handlers, the real DB, the real event loop. They're the acceptance layer.

10.7 What we could add

CI (nothing enforces green on `main`), a migration-vs-model check, property-based tests for the pricing math, and a small load test to *measure* the per-tenant throughput ceiling the `FOR UPDATE` lock imposes (right now we only argue about it qualitatively).

11. Deployment & developer experience

What we did. `docker compose up --build` -> healthchecked Postgres -> api waits for healthy -> `alembic upgrade head` -> `python seed_tenant.py` (idempotent: creates a "Demo Tenant" + Free subscription, prints its id) -> `uvicorn --reload`. Tests: `docker compose run --rm api pytest`. Probes: `docker compose exec api python verify_probes.py`.

Why the healthcheck + `depends_on: service_healthy`. Without it, `alembic upgrade head` can fire before Postgres is accepting connections and the stack dies on the first boot — exactly the "a stranger clones it and it doesn't work" failure the brief warns about.

Why `ENV_VAR: -default` in the compose env list. `ENABLE_SCHEDULER=${ENABLE_SCHEDULER:-true}` etc. If an optional var is unset, compose would otherwise pass an **empty string**, and pydantic parsing `ENABLE_SCHEDULER=""` as a bool raises a `ValidationError` at startup. The `:-default` makes the stack boot with zero host env beyond the three required secrets.

Why `run --rm` for the test command (not `exec`). `exec` needs the container already running; `run --rm` doesn't. The submission manifest's `test: command` is executed directly by the evaluator, possibly without `up` first — so `run --rm` is the safe form. (This was `exec` originally and got fixed in the hardening pass.)

The trade-off. The default is a dev config (`--reload`, `bind-mount`, weak DB creds, fixed port). It's optimised for "clone and explore", not "deploy". A prod compose profile (no reload, no mount, real secrets, an HA database) is the missing piece; it wasn't built because the capstone's bar is demonstrability, not production readiness, and that's stated in README limitations.

12. Scope — what we deliberately left out, and why

Left out	Why	Category
Real auth (API keys / JWT)	explicit non-goal; the hard parts are elsewhere; built in a separate assignment	scope
Proration on mid-cycle plan change	"a stretch goal with real teeth" per the brief	stretch
Invoice generation / PDF / email	invoicing is a stretch goal	stretch
Overage billing (allow + charge beyond limit)	stretch goal; our boundary rule is "hard stop"	stretch
Usage alerts (80% / 100% notifications)	stretch goal	stretch
A real LLM call	brief says meter numbers, not models; adds cost/secret/nondeterminism	scope
More than 2 plans	"2 plans" is the stated scope; the <code>plans</code> table makes adding one an INSERT	scope
A production compose / HA Postgres	capstone bar is "a stranger can run and demo it"	scope
CI/CD	not required; acknowledged gap	gap

The discipline here: every one of these is written down as a **choice** (in `capstone.yaml` `non_goal`, `README` limitations, or `tech-debt-tracker.md`), so it reads as "we decided not to", not "we forgot".

13. Honest retrospective — what we'd change

1. **The FOR UPDATE quota lock's blast radius.** It's correct, but it serialises all of

one tenant's `/generate` traffic through one row lock for the whole request. The better design for a high-QPS tenant is an atomic conditional `UPDATE ... WHERE current + q <= limit RETURNING ...` against a per-(tenant,period) counter row — no long lock, still exactly-once. We'd keep the event log for audit and maintain the counter as a projection. We chose the lock because it's obviously correct and the scope doesn't stress it; we should be able to say exactly when we'd switch. We've now measured the **correctness** side of that trade (bench/race_condition_benchmark.py: 97.5% -> 0% overcount rate across 40 trials) — we have not yet measured the **throughput** side (requests/sec one tenant can push through the locked path), which is exactly the number a "when would you switch" follow-up is really asking for.

2. **The dead type='api_call' path.** Tested, handled everywhere, emitted nowhere.

Either wire a bulk-metering endpoint that uses it, or delete it and simplify `api_calls_used` to `COUNT(*)`.

3. **Migrations aren't in the test loop.** `create_all` in tests + hand-written

migrations = possible silent drift. Add a CI job that runs the migrations and diffs against the model metadata.

4. **In-process scheduler.** Fine for one instance; wrong for many. Move to an external

scheduler or add a leader lock before scaling.

5. **No CI at all.** `capstone.yaml` names the `run/test` commands but nothing runs

them on push.

6. `RollupService` vs `QuotaService` on "no subscription". They **agree in meaning**

(a tenant with no subscription is not an active Free tenant) but **differ in mechanism**: `/generate` returns 402, `/usage` returns 200 with `plan_id="none"`. In normal operation this never triggers (every tenant is seeded with a subscription and the invariant is documented), but a nitpicker will notice `/usage` succeeding for a state `/generate` treats as an error.

7. `seed_tenant.py` sets `current_period_end == current_period_start == now()` — an

already-expired window. Harmless (the code falls back to calendar-month), but sloppy seed data.

8. **The unit is misnamed** — "micro-cents" is really micro-dollars. Kept for

contract/evidence stability; `cost_micros` would be clearer.

14. How to explain this in 30 seconds and in 3 minutes

30 seconds. "It's the backend a SaaS needs for usage-based billing. It meters token usage per tenant into an append-only event log, enforces the plan's monthly quota **before** each call — with a row lock so two concurrent requests can't both slip past the limit — prices usage in exact integer money with four token-category rates, and mirrors Stripe subscription state through signature-verified, deduplicated webhooks. A nightly reconciliation job re-syncs anything the webhooks miss. Every scary case — double charge, replayed webhook, exact boundary, concurrent boundary — has a test that proves it can't happen."

3 minutes. Walk one `POST /generate`: header validation -> idempotency pre-check (a retry returns the original response, no new row) -> `check_quota` takes a `SELECT ... FOR UPDATE` on the tenant's subscription row, rejects with 402 if the plan doesn't allow it or 429 (+Retry-After) if the allowance is used up — **before** anything is written -> `MeterService.record` prices the four token categories separately, inserts one immutable event, and the DB's unique constraint on `(tenant_id, idempotency_key)` is the real guarantee that a concurrent duplicate lands on one row -> commit releases the lock. Then one Stripe upgrade: `/checkout` creates a hosted session with `tenant_id` in the metadata -> the customer pays -> `checkout.session.completed` arrives -> we verify the HMAC over the raw body first (forged -> 400, nothing happens), dedupe on the event id, re-fetch the subscription for authoritative state, and upsert our local mirror -> next `/usage` shows Pro. Then the safety net: nightly at 03:00 UTC, a job lists every Stripe subscription, repairs any local drift, downgrades any tenant whose Stripe subscription is gone, and records the run (with retries and a CRITICAL alert on failure) in a `job_runs` table you can query. The through-line is **correctness by construction** — database constraints and row locks, not hopeful application checks — because in billing a bug is money.

Appendix — stack at a glance

Python 3.12 · FastAPI + Pydantic v2 · async SQLAlchemy 2.0 + asyncpg · PostgreSQL 16 · Alembic (2 migrations) · Stripe test mode (stripe SDK, pinned <13) · APScheduler (pinned <4) · pytest + pytest-asyncio + httpx · Docker Compose. **6 tables · 7 routes · 9 service modules · 7 test modules · 31 tests · 5 Layer-2 probes · 2 reproducible benchmarks** (bench/: 97.5% -> 0% concurrency fix, ~13x index speedup).